TurboHPR: Streaming sEMG-Based Hand Pose Regression with Over 100× Speedup

Danjie Chen*
School of Integrated Circuits
Tsinghua University
Beijing, China
cdj25@mails.tsinghua.edu.cn

Kang Zhao*,†
School of Integrated Circuits
Tsinghua University
Beijing, China
zhaok14@tsinghua.org.cn

Xirui Guo
School of Integrated Circuits
Tsinghua University
Beijing, China
guoxr@mail.tsinghua.edu.cn

Hanjun Jiang
School of Integrated Circuits
Tsinghua University
Beijing, China
jianghanjun@tsinghua.edu.cn

***Abstract*—Continuous sEMG-based hand pose regression is important for wearable human-machine interfaces, prosthetic control, and rehabilitation, enabling natural hand control from non-invasive muscle signals. However, real-time decoding is difficult because many temporal models use long windows, repeatedly processing history and causing redundancy.We propose TurboHPR, a high-speed framework for streaming sEMG-based Hand Pose Regression (HPR). It is based on two insights: long windows are often unnecessary in smooth or predictable motion states, and useful cues in long sEMG contexts are unevenly distributed. TurboHPR treats long-context computation as a conditional resource rather than a fixed cost. It combines a fast residual update path, a compressed-context calibration branch, and a similarity-guided scheduler that decides when calibration is needed.Experiments on the emg2pose dataset show that TurboHPR achieves state-of-the-art performance on cross-user, cross-stage, and cross-user-stage subsets, with an average Angular Error of 0.288 rad and Landmark Distance of 23.6 mm. It also achieves the lowest computational cost among the compared state-of-the-art methods, requiring only 35.7M MACs per inference call. Remarkably, TurboHPR delivers a 121.60× speedup over prior art, making accurate streaming HPR practical under strict latency constraints.**

***Index Terms*—Surface electromyography, hand pose regression, context compression, real-time decoding, mamba.**

I. Introduction

Surface electromyography (sEMG) enables non-invasive hand control for wearable human-machine interaction, prosthetic control, and rehabilitation systems [1,2,3,4,5,6]. Discrete sEMG-based hand gesture recognition has been widely studied for myoelectric control and human-machine interaction [6,7,8,9,10]. Among sEMG-based control tasks, continuous Hand Pose Regression (HPR) is particularly important because it estimates frame-level hand motion rather than discrete gesture labels, allowing more natural and fine-grained control [11,12]. However, deploying sEMG-based HPR in streaming scenarios requires not only accurate decoding but also fast inference under strict latency constraints [4,13].

A common way to improve HPR accuracy is to use temporal models with long input windows, including temporal convolutional, Transformer-based, and state-space sequence models [13,14,15,16,17,18,19,20,21,22,23,24]. Long-context modeling can provide useful historical cues for stabilizing pose estimation, but repeatedly processing the full history at every streaming step introduces substantial redundant computation and latency [25,26,27,28,29,30]. This limits the practicality of existing long-context decoders for real-time HPR, where predictions must be updated continuously and efficiently.

To address this limitation, we propose *TurboHPR*, a high-speed framework for streaming sEMG-based Hand Pose Regression. The key idea of TurboHPR is to make long-context computation conditional rather than fixed. This design is motivated by two insights. First, many streaming states are locally predictable, where recent sEMG packets and previous pose estimates are sufficient for reliable updates. Second, not all frames in a long sEMG window contribute equally to the current pose prediction; some provide useful calibration cues, while others are largely redundant.

TurboHPR accelerates streaming HPR through three coordinated components: a fast residual update path, a compressed-context calibration branch, and a similarity-guided turbo scheduler. For stable signal states, TurboHPR performs lightweight residual updates to avoid unnecessary long-context computation. When the signal changes or local prediction becomes unreliable, it invokes compressed-context calibration to correct the pose estimate using compact historical information. In this way, TurboHPR preserves the benefit of long-context modeling while substantially reducing its streaming cost, as shown in Fig. 1.

We evaluate TurboHPR on the emg2pose dataset [12] across the cross-user, cross-stage, and cross-user-stage test subsets. TurboHPR achieves an average Angular Error (AE) of 0.288 rad and an average Landmark Distance (LD) of 23.6 mm. Meanwhile, it reduces inference latency to 1.106 ms per update and achieves 121.60× speedup over the slowest compared method, demonstrating its effectiveness for fast streaming HPR.

The main contributions are summarized as follows:

- We identify redundant long-context computation as a major bottleneck for real-time sEMG-based HPR.
- We propose *TurboHPR*, a high-speed streaming HPR framework that treats long-context computation as an adaptive calibration resource.

*Equal contribution.
†Corresponding authors.

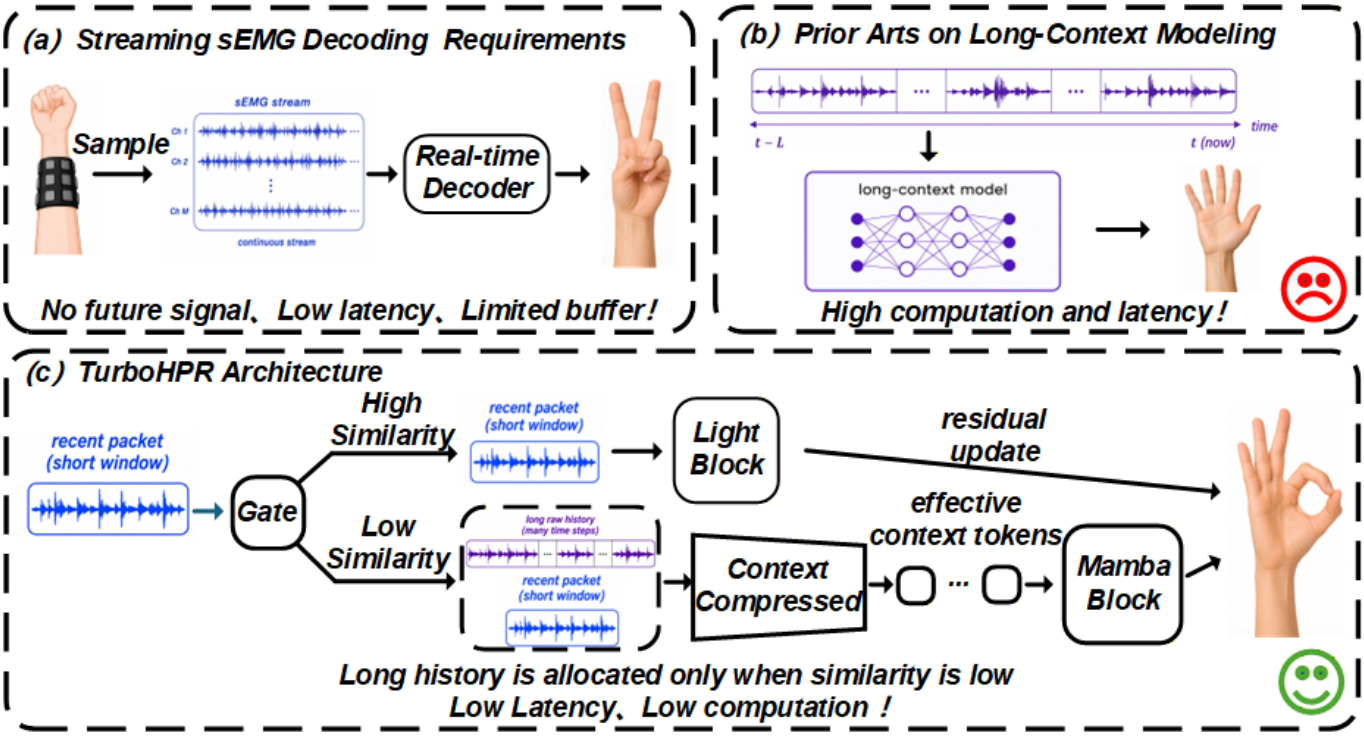


Fig. 1. Motivation and overview of TurboHPR. (a) Streaming sEMG-based HPR requires causal and low-latency prediction. (b) Existing long-context decoders repeatedly process historical windows, causing redundant computation. (c) TurboHPR accelerates HPR by adaptively switching between fast residual update and compressed-context calibration.

- We design a turbo scheduling mechanism that switches between fast residual update and compressed-context calibration according to packet-level similarity.
- Experiments on emg2pose show that TurboHPR achieves strong HPR accuracy across multiple generalization test subsets while providing over 100× streaming speedup.

II. METHODS

A. Problem Formulation and Streaming Constraint

We formulate sEMG-based Hand Pose Regression (HPR) as a causal streaming prediction problem. Given an sEMG stream

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\} \quad \mathbf{x}_t \in \mathbb{R}^C \tag{1}$$

the goal is to estimate the hand pose sequence

$$\mathbf{Y} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_T\} \quad \mathbf{y}_t \in \mathbb{R}^D \tag{2}$$

During streaming inference, the prediction at time t must depend only on current and past sEMG observations:

$$\hat{\mathbf{y}}_t = F(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_t) \tag{3}$$

At each update step, TurboHPR uses a recent packet and an optional historical context:

$$\mathbf{P}_t = \mathbf{X}_{t-w+1:t}, \quad \mathbf{H}_t = \mathbf{X}_{t-w-L+1:t-w} \tag{4}$$

where w is the packet length, L is the history length, and $L \gg w$. The recent packet $\mathbf{P}_t$ supports fast local prediction, while $\mathbf{H}_t$ is used only when long-context calibration is needed.

The streaming update interval is

$$T_{\text{update}} = \frac{\text{Stride}}{f_s} \tag{5}$$

where Stride is the number of newly arrived sEMG samples and f_s is the sampling frequency. To satisfy real-time HPR, the inference latency should meet

$$\text{Latency}_{\text{method}} < T_{\text{update}}, \quad T_{\text{update}} < T_{\text{threshold}} \tag{6}$$

where $T_{\text{threshold}}$ denotes the smooth-control latency threshold. TurboHPR is designed to reduce redundant long-context computation while maintaining accurate causal HPR.

B. TurboHPR Framework

TurboHPR accelerates streaming HPR by treating long-context modeling as an adaptive calibration resource rather than a fixed cost. It contains three components: a fast residual update path, a compressed-context calibration branch, and a similarity-guided turbo scheduler. For locally stable sEMG states, TurboHPR uses the fast path for low-latency updates. When the signal changes or local prediction becomes unreliable, it invokes the calibration path to correct the prediction with compact historical information. This design is also related to dynamic inference and large-small model collaboration, where lightweight computation handles easy states and expensive modules are invoked only when additional correction is needed [29,30,31,32,33,34,35,36].

C. Fast Residual Update Path

The fast path estimates the current pose by updating the previous prediction. Given the recent packet $\mathbf{P}_t$, it predicts a residual pose change:

$$\Delta\hat{\mathbf{y}}_t = f_{\text{fast}}(\mathbf{P}_t) \tag{7}$$

where $f_{\text{fast}}(\cdot)$ is implemented as a lightweight MLP that maps the recent packet to a pose residual. The fast-path prediction is

$$\hat{\mathbf{y}}_t^f = \hat{\mathbf{y}}_{t-1} + \Delta\hat{\mathbf{y}}_t \tag{8}$$

This path exploits temporal continuity and avoids repeated long-window inference during stable streaming states. It follows the general principle of lightweight inference for resource-constrained deployment [37].

D. Compressed-Context Calibration Branch

When long-context correction is required, TurboHPR builds a calibration context by combining the historical context and the current packet:

$$\mathbf{C}_t = [\mathbf{H}_t, \mathbf{P}_t] \tag{9}$$

The context is divided into a far-context segment and a recent-context segment:

$$\mathbf{C}_t^{\text{far}} = \mathbf{X}_{t-w-L+1:t-L_r}, \quad \mathbf{C}_t^{\text{recent}} = \mathbf{X}_{t-L_r+1:t} \tag{10}$$

where L_r is the recent-context length.

Instead of processing the full far context, TurboHPR compresses it using learnable-query attention:

$$\begin{aligned} \mathbf{Z}_t &= g_{\text{comp}}(\mathbf{C}_t^{\text{far}}) = \text{Attn}(\mathbf{Q}_c, \mathbf{K}_t, \mathbf{V}_t) \\ \mathbf{Z}_t &\in \mathbb{R}^{N_c \times d}, \quad N_c \ll |\mathbf{C}_t^{\text{far}}| \end{aligned} \tag{11}$$

Here, $\mathbf{Q}_c$ denotes learnable queries, while $\mathbf{K}_t$ and $\mathbf{V}_t$ are generated from $\mathbf{C}_t^{\text{far}}$. N_c and d denote the number and dimension of compressed context tokens, respectively. The calibrated pose is then predicted as

$$\hat{\mathbf{y}}_t^c = f_{\text{calib}}(\mathbf{Z}_t, \mathbf{C}_t^{\text{recent}}) \tag{12}$$

Thus, the calibration branch preserves useful long-range information while reducing the cost of raw long-context modeling.

E. Similarity-Guided Turbo Scheduling

TurboHPR selects the fast path or calibration branch according to packet-level similarity. Given the current and previous packets, the similarity score is

$$s_t = \text{CC}(\mathbf{P}_t, \mathbf{P}_{t-1}) \tag{13}$$

where $\text{CC}(\cdot)$ denotes a cosine similarity measure. The final prediction is

$$\hat{\mathbf{y}}_t = \begin{cases} \hat{\mathbf{y}}_t^f & \text{if } \hat{\mathbf{y}}_{t-1} \text{ exists and } s_t \geq \tau \\ \hat{\mathbf{y}}_t^c & \text{otherwise} \end{cases} \tag{14}$$

where τ is the similarity threshold. In this way, TurboHPR performs fast residual updates for stable states and invokes long-context calibration only when necessary.

III. EXPERIMENTAL RESULTS

A. Experimental Setup

We evaluate *TurboHPR* on the emg2pose dataset [12] for streaming sEMG-based Hand Pose Regression (HPR). The test-set splits, sample counts, preprocessing pipeline, and evaluation metrics follow the official emg2pose configuration. Specifically, we evaluate generalization on three test subsets: cross-user, cross-stage, and cross-user-stage, and measure HPR accuracy using Angular Error (AE, rad) and Landmark Distance (LD, mm). The recent packet length is set to $w = 16$, and the historical context length is set to $L = 1280$.

Streaming efficiency. Streaming efficiency is evaluated by latency and speedup:

$$\text{Speedup} = \frac{\max_i \text{Latency}_i}{\text{Latency}_{\text{method}}} \tag{15}$$

where Latency_i is the latency of the i-th compared method. All latency results are measured on a single NVIDIA A100-SXM4-80GB GPU. To faithfully emulate online streaming deployment, all evaluations are conducted under a streaming inference protocol with batch size $\text{bs} = 1$, where each update processes one newly arrived sEMG packet without batching across samples or future time steps.

B. Comparison With State-of-the-Art Methods

Table I compares TurboHPR with representative sEMG-based HPR methods on three scenario-specific test sets. TurboHPR achieves the lowest AE and LD on all three test sets, showing strong HPR accuracy across user, stage, and user-stage scenarios.

TABLE I
ACCURACY COMPARISON ON EMG2POSE.

Test Set	Method	AE (rad) ↓	LD (mm) ↓
User	Emg2Pose [12]	0.316 ± 0.005	25.3 ± 0.3
	NeuroPose [11]	0.320 ± 0.007	26.5 ± 0.7
	OnlineTransformer [13]	0.291 ± 0.003	24.5 ± 0.2
	NaiveMamba [19]	0.270 ± 0.003	22.0 ± 0.3
	Ours	$\mathbf{0.258 \pm 0.001}$	$\mathbf{20.4 \pm 0.1}$
Stage	Emg2Pose [12]	0.364 ± 0.007	29.1 ± 0.3
	NeuroPose [11]	0.356 ± 0.006	29.3 ± 0.8
	OnlineTransformer [13]	0.331 ± 0.002	28.3 ± 0.2
	NaiveMamba [19]	0.321 ± 0.005	27.1 ± 0.5
	Ours	$\mathbf{0.305 \pm 0.001}$	$\mathbf{25.1 \pm 0.2}$
User & Stage	Emg2Pose [12]	0.363 ± 0.008	29.4 ± 0.2
	NeuroPose [11]	0.352 ± 0.009	29.4 ± 0.8
	OnlineTransformer [13]	0.322 ± 0.002	28.4 ± 0.2
	NaiveMamba [19]	0.317 ± 0.006	27.4 ± 0.6
	Ours	$\mathbf{0.304 \pm 0.001}$	$\mathbf{25.7 \pm 0.2}$

For streaming efficiency evaluation, the update interval T_{update} is determined by the stride used by each method. Emg2Pose and NeuroPose are evaluated using their official stride configurations. For OnlineTransformer, its original stride is 7, corresponding to an update interval of 3.5 ms. However, under this setting, the measured inference latency is over 130 ms, which is much larger than T_{update} and therefore violates the streaming constraint. To provide a feasible streaming

evaluation, we enlarge its stride to 400, corresponding to $T_{\text{update}} = 200$ ms, so that Latency $< T_{\text{update}}$. However, this update interval is still larger than the smooth-control threshold $T_{\text{threshold}}$, and thus OnlineTransformer is marked as streaming but not smooth.

TABLE II
STREAMING SPEED COMPARISON.

Method	MACs ↓	Lat. ↓	T_{update} ↓	Spd. ↑	Str.&Sm.
Emg2Pose	522.1M	30.4	2895	4.43×	No
NeuroPose	220.6M	4.5	2000	29.92×	No
OnlineTrans.	61.1M	134.5	200	1.00×	No
NaiveMamba	274.4M	12.3	25	10.92×	Yes
Ours	**35.7M**	**1.1**	**8**	**121.60×**	**Yes**

Note: OnlineTrans. denotes OnlineTransformer. Lat. and T_{update} are in milliseconds, and Spd. is relative to the slowest method. Str./Sm. indicates Latency $< T_{\text{update}} < 40$ ms (25 Hz).

Beyond latency speedup, TurboHPR also achieves the lowest computational cost, requiring only 35.7M MACs per inference call. This reduced computation, together with the 1.1 ms latency, provides a larger timing margin for streaming sEMG decoding and makes TurboHPR more suitable for low-compute edge devices.

C. Ablation Study

To evaluate each component, we compare TurboHPR with three ablated variants: light-only, calibration-only, and w/o context compression.

TABLE III
ABLATION STUDY OF TURBOHPR.

Method	AE ↓	LD ↓	MACs ↓	Lat. ↓	Spd. ↑
Light-only	1.595 ± 0.025	64.9 ± 1.3	51.7 K	0.3	446.88×
calibration-only	0.299 ± 0.043	24.9 ± 4.2	151.1 M	3.3	41.37×
w/o Context Compression	0.385 ± 0.023	30.3 ± 1.2	103.9 M	2.0	67.18×
Ours	$\mathbf{0.305 \pm 0.044}$	$\mathbf{25.4 \pm 4.2}$	**35.7 M**	**1.0**	**134.50×**

Note: The ablation study is conducted on the mini subset of the emg2pose dataset.

Table III shows that TurboHPR balances the two branches: the lightweight branch provides low latency but accumulates errors, while the calibration branch improves accuracy at higher cost. Adaptive switching preserves near-calibration accuracy with a 3.25× speedup.

D. Context Analysis

To analyze how useful information is distributed in long temporal context, we perform prefix occlusion by progressively masking the oldest frames in the 1280-frame history and retaining only the most recent frames for decoding.

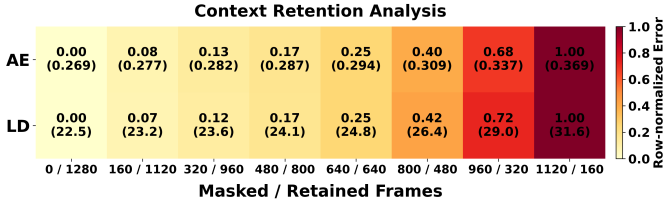


Fig. 2. Context retention analysis under prefix occlusion.

As shown in Fig. 2, the error grows non-uniformly as the retained context is reduced. Removing part of the distant history causes only moderate degradation, while further reducing the retained context leads to a sharp increase in AE and LD. This indicates that useful cues are unevenly distributed across the long sEMG context, motivating TurboHPR to compress historical information into compact context tokens rather than repeatedly processing the full raw window.

E. Effect of Similarity Threshold

Fig. 3 shows the effect of the similarity threshold on prediction accuracy and execution load. As the threshold increases, both AE and LD decrease rapidly and then become stable. Meanwhile, the execution load gradually shifts from the Light Load branch to the Calibration Load branch, indicating a trade-off between accuracy and computational cost.

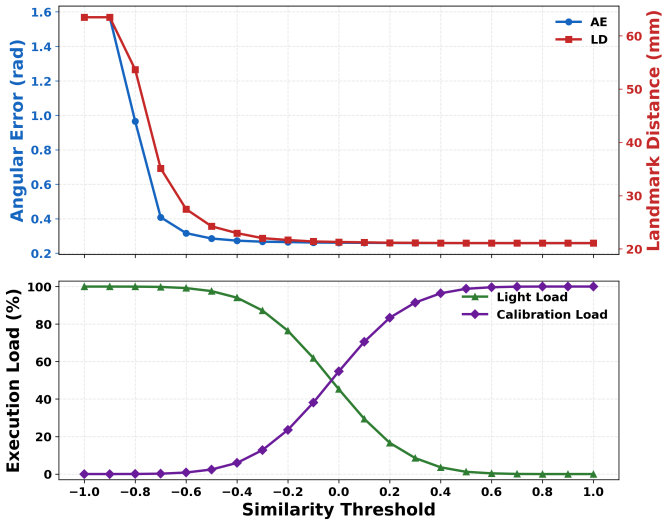


Fig. 3. Effect of the similarity threshold on AE, LD, and execution load.

IV. CONCLUSION

In this work, we proposed *TurboHPR*, a real-time framework for continuous sEMG-based hand pose regression. The method combines lightweight residual update with compressed-context calibration to avoid unnecessary long-window modeling while preserving historical correction when needed. Experiments on the emg2pose dataset show that the proposed method achieves strong performance across the cross-user, cross-stage, and cross-user-stage test subsets with substantially improved inference efficiency. Ablation and context retention analyses further validate the effectiveness of lightweight update, context compression, and adaptive long-context allocation. These results suggest that adaptive compressed-context extraction is a promising direction for accurate and low-latency streaming sEMG decoding, especially for wearable human-machine interfaces and prosthetic control. Future work will explore deployment on edge devices and further improve robustness under long-term use.

REFERENCES

[1] R. Merletti and D. Farina, Eds., *Surface Electromyography: Physiology, Engineering, and Applications*. Wiley-IEEE Press, 2016.

[2] M. Atzori, A. Gijsberts, I. Kuzborskij, S. Elsig, A.-G. M. Hager, O. Deriaz, C. Castellini, H. Muller, and B. Caputo, "The ninapro database: A resource for semg naturally controlled robotic hand prosthetics," *Frontiers in Neurorobotics*, vol. 9, 2015.

[3] D. Xiong, D. Zhang, X. Zhao, and Y. Zhao, "Deep learning for emg-based human-machine interaction: A review," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 3, pp. 512–533, 2021.

[4] S. Jiang, B. Lv, W. Guo, C. Zhang, H. Wang, X. Sheng, and P. B. Shull, "Feasibility of wrist-worn, real-time hand, and surface gesture recognition via semg and imu sensing," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 8, pp. 3376–3385, 2018.

[5] S. Pancholi and A. M. Joshi, "Portable emg data acquisition module for upper limb prosthesis application," *IEEE Sensors Journal*, vol. 18, no. 8, pp. 3436–3443, 2018.

[6] U. Côté-Allard, C. L. Fall, A. Drouin, A. Campeau-Lecours, C. Gosselin, K. Glette, F. Laviolette, and B. Gosselin, "Deep learning for electromyographic hand gesture signal classification using transfer learning," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27, no. 4, pp. 760–771, 2019.

[7] W. Wei, Q. Dai, Y. Wong, Y. Hu, M. Kankanhalli, and W. Geng, "Surface-electromyography-based gesture recognition by multi-view deep learning," *IEEE Transactions on Biomedical Engineering*, vol. 66, no. 10, pp. 2964–2973, 2019.

[8] Y. Hu, Y. Wong, W. Wei, Y. Du, M. Kankanhalli, and W. Geng, "A novel attention-based hybrid cnn-rnn architecture for semg-based gesture recognition," *IEEE Access*, vol. 7, pp. 92 065–92 076, 2019.

[9] S. Padhy, "A tensor-based approach using multilinear svd for hand gesture recognition from semg signals," *IEEE Sensors Journal*, vol. 21, no. 5, pp. 6634–6642, 2021.

[10] U. Côté-Allard, G. Gagnon-Turcotte, A. Phinyomark, K. Glette, E. J. Scheme, F. Laviolette, and B. Gosselin, "Unsupervised domain adversarial self-calibration for electromyography-based gesture recognition," *IEEE Access*, vol. 8, pp. 177 941–177 955, 2020.

[11] Y. Liu, S. Zhang, and M. Gowda, "Neuropose: 3d hand pose tracking using emg wearables," in *Proceedings of the Web Conference 2021*, 2021, pp. 1471–1482.

[12] S. Salter, R. Warren, C. Schlager, A. Spurr, S. Han, R. Bhasin, Y. Cai, P. Walkington, A. Bolarinwa, R. Wang *et al.*, "emg2pose: A large and diverse benchmark for surface electromyographic hand pose estimation," in *The Thirty-eighth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

[13] N. Leroux, J. Finkbeiner, and E. Neftci, "Online transformers with spiking neurons for fast prosthetic hand control," in *2023 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 2023, pp. 1–6.

[14] P. Tsinganos, B. Cornelis, J. Cornelis, B. Jansen, and A. Skodras, "Improved gesture recognition based on semg signals and tcn," in *2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 1169–1173.

[15] J. L. Betthauser, C. L. Hunt, L. E. Osborn, R. R. Kaliki, and N. V. Thakor, "Stable electromyographic sequence prediction during movement transitions using temporal convolutional networks," in *2019 9th International IEEE/EMBS Conference on Neural Engineering (NER)*, 2019, pp. 1046–1049.

[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[17] M. Montazerin, E. Rahimian, F. Naderkhani, S. F. Atashzar, S. Yanushkevich, and A. Mohammadi, "Transformer-based hand gesture recognition from instantaneous to fused neural decomposition of high-density emg signals," *Scientific Reports*, vol. 13, no. 1, p. 11000, 2023.

[18] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," *arXiv preprint arXiv:2111.00396*, 2021.

[19] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Conference on Language Modeling*, 2024.

[20] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," *arXiv preprint arXiv:2405.21060*, 2024.

[21] O. Lieber, B. Lenz, H. Bata, G. Cohen, J. Osin, I. Dalmedigos, E. Safahi, S. Meirom, Y. Belinkov, S. Shalev-Shwartz *et al.*, "Jamba: A hybrid transformer-mamba language model," *arXiv preprint arXiv:2403.19887*, 2024.

[22] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, "Vmamba: Visual state space model," *arXiv preprint arXiv:2401.10166*, 2024.

[23] A. Hatamizadeh and J. Kautz, "Mambavision: A hybrid mamba-transformer vision backbone," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 25 261–25 270.

[24] W. Yu and X. Wang, "Mambaout: Do we really need mamba for vision?" in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[25] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are rnns: Fast autoregressive transformers with linear attention," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 5156–5165.

[26] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," *arXiv preprint arXiv:2006.04768*, 2020.

[27] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, "Rethinking attention with performers," in *International Conference on Learning Representations*, 2021.

[28] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *International Conference on Learning Representations*, 2023.

[29] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "Model compression and acceleration for deep neural networks: The principles, progress, and challenges," *IEEE Signal Processing Magazine*, vol. 35, no. 1, pp. 126–136, 2018.

[30] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022.

[31] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[32] A. Graves, "Adaptive computation time for recurrent neural networks," *arXiv preprint arXiv:1603.08983*, 2016.

[33] L. Wang and K.-J. Yoon, "Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 6, pp. 3048–3068, 2022.

[34] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.

[35] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *International Conference on Learning Representations*, 2018.

[36] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, "Deebert: Dynamic early exiting for accelerating bert inference," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 2246–2251.

[37] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.